



# APIs de IA — Conceptos Clave y Llamadas



▢ Módulo 4 · Introducción a APIs de IA

## APIs de IA — Conceptos Clave y Primeras Llamadas

Ejercicio de Clase · Ejercicio 2 de 3 · Bloques 4.3 — 4.5 — 4.6 — 4.7

|                                 |                 |
|---------------------------------|-----------------|
| Nombre del alumno/a             | Angel Echenique |
| Fecha                           | 10/04/2026      |
| ▢ Enlace GitHub del repositorio |                 |

▢ **Instrucciones:** Aplica los conceptos clave vistos en clase. Este ejercicio combina teoría y práctica. Analiza código, diseña prompts y toma decisiones técnicas. ¡No hay que escribir código desde cero, pero sí entender y razonar sobre él!

### BLOQUE 1 — Conceptos clave aplicados (4.3)

Vimos que tokens, context window, temperature y system prompt son los parámetros clave de cualquier llamada a una API de IA. Para cada escenario, identifica qué parámetro hay que ajustar y cómo:

| Escenario / Problema                                                                             | ¿Qué parámetro ajustas? | ¿Cómo lo ajustas?                                                                                                       |
|--------------------------------------------------------------------------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Tu chatbot genera respuestas demasiado largas y consumes muchos tokens.                          | max_tokens              | Reducir el valor (ej: de 1000 a 300) limita la longitud de la respuesta generada.                                       |
| La IA responde de forma demasiado creativa y sus respuestas varían mucho ante la misma pregunta. | Temperature             | Bajarla a <b>0.0 - 0.3</b> para obtener respuestas más predecibles y consistentes.                                      |
| Tu modelo 'olvida' el principio de la conversación cuando el historial es muy largo.             | Context window          | Limitar el historial de conversación a menos tokens que el límite del modelo (ej: truncando los mensajes más antiguos). |



|                                                                                         |               |                                                                                                   |
|-----------------------------------------------------------------------------------------|---------------|---------------------------------------------------------------------------------------------------|
| Quieres que la IA actúe siempre como un asistente técnico experto en Python.            | System prompt | Añadir un mensaje de sistema con el rol: "Eres un experto en Python."                             |
| Los tests generados por la IA no siguen el formato de tu proyecto (pytest, sin clases). | System prompt | Especificar en el prompt de sistema: "Genera tests usando pytest, sin clases, funciones sueltas." |
| Necesitas buscar documentos por significado, no por palabras exactas.                   | Embeddings    | Usar la API de embeddings para convertir textos a vectores y buscar por similitud semántica.      |

## 1.2 — System Prompt: diseña el comportamiento

Diseña un system prompt profesional para cada uno de estos asistentes. Recuerda que debe incluir: rol, tono, reglas y formato de respuesta:

| Asistente                                                                      | Tu system prompt                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Un asistente técnico experto en APIs de IA para desarrolladores junior.</b> | Eres un experto técnico en APIs de IA.<br>Tu misión es enseñar a desarrolladores junior de forma clara y didáctica.<br>Usa ejemplos prácticos y código funcional.<br>Si no sabes algo, dilo claramente.<br>Evita tecnicismos innecesarios.                                            |
| <b>Un clasificador de tickets de soporte que responde SOLO en JSON.</b>        | Eres un sistema de clasificación de tickets de soporte.<br>Debes responder ÚNICAMENTE en formato JSON válido con los campos: categoria (tecnico/facturacion/cuenta), prioridad (alta/media/baja), departamento (soporte/facturacion/desarrollo).<br>No escribas texto fuera del JSON. |
| <b>Un tutor de programación que explica conceptos con analogías simples.</b>   | Eres un tutor de programación pacientes y cercano.<br>Explica conceptos de programación usando analogías de la vida cotidiana. Cuando introduces un concepto nuevo, usa ejemplos prácticos y compara con situaciones que el estudiante pueda entender. Anima a hacer preguntas.       |



## 1.3 — Few-shot Learning: enseña con ejemplos

Vimos que el few-shot learning consiste en incluir ejemplos en el prompt para que el modelo aprenda el patrón. Completa la tabla indicando los ejemplos que añadirías al prompt y qué patrón quieres que aprenda:

| Tarea que quieres que la IA haga                                         | 2-3 ejemplos que incluirías en el prompt                                                                                                                                                                                                                                              | Patrón que aprende el modelo                                                                                                  |
|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Clasificar reseñas de productos como POSITIVO / NEUTRO / NEGATIVO.       | <p><b>Ejemplo 1:</b> "Producto increíble, lo recomiendo" →POSITIVO</p> <p><b>Ejemplo 2:</b> "Es correcto, sin más" → NEUTRO</p> <p><b>Ejemplo 3:</b> "No funcionó, devolvería el dinero" →NEGATIVO</p>                                                                                | Aprende a identificar palabras clave de opinión (increíble/recomiendo = positivo; correcto = neutro; no funcionó = negativo). |
| Extraer el nombre y el email de textos en lenguaje natural.              | <p><b>Ejemplo 1:</b> "Me llamo Carlos y mi email es carlos@email.com" → {"nombre": "Carlos", "email": "carlos@email.com"}</p> <p><b>Ejemplo 2:</b> "Soy María García, puedes contactarme en maria.garcia@test.org" → {"nombre": "María García", "email": "maria.garcia@test.org"}</p> | Aprende a reconocer patrones de nombre + email en diferentes formatos de texto.                                               |
| Traducir frases técnicas de inglés a español manteniendo el tono formal. | <p><b>Ejemplo 1:</b> "The API processes requests in real-time." →"La API procesa solicitudes en tiempo real."</p> <p><b>Ejemplo 2:</b> "Machine learning models require training data." → "Los modelos de aprendizaje automático requieren datos de entrenamientos"</p>               | Aprende a mantener la terminología técnica y el registro formal en español.                                                   |



## APIs de IA — Conceptos Clave y Llamadas



En clase vimos que el coste de las APIs de IA se calcula en tokens. Responde las siguientes preguntas:

| Pregunta                                                                              | Tu respuesta                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ¿Qué modelo recomendarías para una app educativa de prototipo y por qué?              | <b>GPT-4o-mini</b> . Porque tiene el mejor equilibrio precio/rendimiento: cuesta hasta 20 veces menos que GPT-4o, es rápido y suficientemente potente para tareas educativas como chatbots, resúmenes simples, clasificación de textos                                                                                                                                                                                              |
| ¿Cuál es el modelo más económico de Anthropic Claude y para qué es ideal?             | <b>Claude 3 Haiku</b> (\$0.25/1M tokens). Es ideal para tareas masivas de bajo coste: clasificación de muchos documentos, procesamiento de grandes volúmenes de texto donde no se necesita razonamiento profundo.                                                                                                                                                                                                                   |
| ¿Qué modelo de Google destacó en visión + texto como alternativa a GPT-4o?            | <b>Gemini 1.5 Pro</b> . Ofrece capacidades multimodales (texto + imagen) similares a GPT-4o, con la ventaja adicional de una ventana de contexto de hasta 2 millones de tokens.                                                                                                                                                                                                                                                     |
| ¿Por qué Gemini 1.5 Flash es ideal para apps que requieren respuestas en tiempo real? | Porque está <b>optimizado para velocidad</b> : es muy rápido (baja latencia) y tiene un coste muy bajo (incluso gratis con límites). Ideal para aplicaciones que necesitan respuesta inmediata como chatbots en vivo, validación de formularios, análisis rápido de imágenes.                                                                                                                                                       |
| Nombra 3 estrategias concretas para reducir el coste de llamadas a la API.            | <ol style="list-style-type: none"><li>1. <b>Usar modelos mini</b> (GPT-4o-mini, Haiku) para tareas simples.</li><li>2. <b>Limitar max_tokens</b> (no generar 1000 tokens si con 200 es suficiente).</li><li>3. <b>Cachear respuestas</b> de preguntas repetidas.</li><li>4. <b>Comprimir prompts</b> (reducir espacios, usar nombres cortos).</li><li>5. <b>Establecer límites mensuales</b> en la consola del proveedor.</li></ol> |

### 2.2 — Calcula el coste real



## APIs de IA — Conceptos Clave y Llamadas



Usando lo aprendido en clase, calcula el coste aproximado de las siguientes aplicaciones. Muestra tu razonamiento:

| Aplicación                                                                                                                                              | Cálculo paso a paso                                                                                                                                                                                                                                                                | Coste estimado                                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chatbot que recibe 100 consultas/día. Cada consulta: 200 tokens input + 300 tokens output. Modelo: GPT-4o-mini (\$0.002/1K tokens).                     | <ul style="list-style-type: none"><li>➤ Tokens input diarios = <math>100 \times 200 = 20.000</math> tokens</li><li>➤ Tokens output diarios = <math>100 \times 300 = 30.000</math> tokens</li><li>➤ Tokens totales diarios = <math>20.000 + 30.000 = 50.000</math> tokens</li></ul> | <ul style="list-style-type: none"><li>➤ Coste diario = <math>50.000 \text{ tokens} \times \\$0.000002 = \\$0.10</math></li><li>➤ Coste mensual = <math>\\$0.10 \times 30 \text{ días} = \\$3.00</math></li></ul>                       |
| Sistema que resume artículos. Procesa 50 artículos/día de 1.500 tokens cada uno, genera resúmenes de 300 tokens. Modelo: Gemini Flash (muy bajo coste). | <ul style="list-style-type: none"><li>➤ Tokens input diarios = <math>50 \times 1.500 = 75.000</math> tokens</li><li>➤ Tokens output diarios = <math>50 \times 300 = 15.000</math> tokens</li><li>➤ Tokens totales diarios = <math>75.000 + 15.000 = 90.000</math> tokens</li></ul> | <ul style="list-style-type: none"><li>➤ Coste diario = <math>90.000 \times \\$0.00000001 = \\$0.0009</math> (menos de 1 céntimo)</li><li>➤ Coste mensual = <math>\\$0.0009 \times 30 = \\$0.027</math> (menos de 3 céntimos)</li></ul> |



## APIs de IA — Conceptos Clave y Llamadas



### BLOQUE 3 — Análisis de llamadas a la API (4.6 — 4.7)

Analiza el siguiente código de llamada a la API de OpenAI en Python. Responde las preguntas sin escribir código nuevo:

```
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
        {'role': 'system', 'content': 'Eres un asistente experto en APIs.'},
        {'role': 'user', 'content': '¿Qué es un token?'}
    ],
    temperature=0.3,
    max_tokens=500
)

print(response.choices[0].message.content)
```

| Pregunta sobre el código                                                                             | Tu respuesta                                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ¿Por qué se usa <code>os.getenv('OPENAI_API_KEY')</code> en lugar de escribir la clave directamente? | Por <b>seguridad</b> . Si escribieras la clave directamente en el código, quedaría expuesta en el repositorio. Usar variables de entorno permite mantener las claves fuera del código fuente. Se almacenan en el sistema operativo o en archivos <code>.env</code> que se ignoran en Git. |
| ¿Qué hace el mensaje con role 'system'? ¿Por qué es diferente al 'user'?                             | El mensaje <code>system</code> define el <b>comportamiento global</b> del modelo: su rol, tono, reglas. Se envía una sola vez al inicio y afecta a TODAS las respuestas. El mensaje <code>user</code> es la <b>pregunta concreta</b> del usuario en esa interacción.                      |
| ¿Qué efecto tiene <code>temperature=0.3</code> en las respuestas del modelo?                         | Con <b>0.3</b> las respuestas son <b>moderadamente consistentes</b> ; no varían mucho ante la misma pregunta, pero tampoco son enfáticamente idénticas. Es el rango ideal para tareas que requieren equilibrio entre precisión y naturalidad.                                             |



|                                                                                                      |                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>¿Qué significa <code>max_tokens=500</code>?<br/>¿Afecta al input o al output?</p>                 | <p><code>max_tokens</code> limita la <b>longitud de la respuesta generada</b> (output). No afecta al input. Significa que el modelo puede generar hasta 500 tokens como respuesta. Si quieres respuestas más cortas, reduces este número</p>                                          |
| <p>¿Qué devuelve <code>response.choices[0].message.content</code>?</p>                               | <p>Devuelve el <b>texto de la respuesta</b> generada por el modelo. <code>response</code> es el objeto completo, <code>choices[0]</code> es la primera opción (siempre hay al menos una), <code>message.content</code> es el texto legible.</p>                                       |
| <p>¿Cómo añadirías el historial de conversación para que el modelo recuerde mensajes anteriores?</p> | <p>Añadiendo todos los mensajes anteriores al</p> <pre>array messages: messages=[ {'role': 'system', 'content': '...'}, {'role': 'user', 'content': 'primer mensaje'}, {'role': 'assistant', 'content': 'primera respuesta'}, {'role': 'user', 'content': 'segunda pregunta'} ]</pre> |

## 3.2 — JavaScript vs Python: diferencias prácticas

En clase vimos la misma llamada en Node.js y Python. Completa la tabla comparativa:

| Aspecto | JavaScript (Node.js) | Python |
|---------|----------------------|--------|
|---------|----------------------|--------|



## APIs de IA — Conceptos Clave y Llamadas



|                                                           |                                                                                                  |                                                                                                     |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Forma de importar el cliente OpenAI                       | <code>import OpenAI from "openai";</code>                                                        | <code>from openai import OpenAI</code>                                                              |
| Gestión de variables de entorno                           | <code>process.env.OPENAI_API_KEY</code>                                                          | <code>os.getenv("OPENAI_API_KEY")</code>                                                            |
| ¿Requiere async/await?                                    | Si, siempre. Las llamadas son asíncronas: <code>await client.chat.completions.create(...)</code> | No requiere por defecto (bloqueante). Opcional con <code>async/await</code> si quieres no bloquear. |
| Extracción de la respuesta del modelo                     | <code>response.choices[0].message.content</code> (igual que Python)                              | <code>response.choices[0].message.content</code>                                                    |
| ¿Para qué entorno es más habitual?                        | Frontend web, apps Node.js, APIs REST en Express/Fastify                                         | Scripts, backend, ciencia de datos, automatizaciones, bots                                          |
| ¿Se usa el mismo endpoint <code>chat.completions</code> ? | Si, exactamente el mismo. La API de OpenAI es independiente del lenguaje.                        | Si, exactamente el mismo                                                                            |

### BLOQUE 4 — Integrando APIs de IA con lo que ya sabes (Pipeline + Modularización)

Ya conoces GitHub Actions y la modularización de código. Ahora conéctalo con las APIs de IA:

|                         |              |
|-------------------------|--------------|
| Pregunta de integración | Tu respuesta |
|-------------------------|--------------|



## APIs de IA — Conceptos Clave y Llamadas



|                                                                                                  |                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ¿Dónde guardarías la API key de OpenAI en un repositorio de GitHub? ¿Por qué no en el código?    | Se guarda en <b>GitHub Secrets</b> (Settings → Secrets and variables → Actions). No debe estar en el código porque el código se sube a GitHub y sería visible para cualquiera. Las API keys son credenciales sensibles que nunca deben exponerse en el repositorio.                               |
| ¿Cómo pasarías la API key de GitHub Secrets a tu script Python en un workflow de GitHub Actions? | Se pasa como variable de entorno usando env: OPENAI_API_KEY: \${{ secrets.OPENAI_API_KEY }} en el workflow, y en Python se accede con <code>os.getenv("OPENAI_API_KEY")</code> .                                                                                                                  |
| Si modularizas tu app con un módulo separado para las llamadas a la API, ¿qué ventajas tiene?    | 1) <b>Separación de responsabilidades</b> : el código de IA está aislado del resto. 2) <b>Reutilización</b> : puedes importar el módulo en diferentes partes. 3) <b>Mantenimiento</b> : cambios solo en un lugar. 4) <b>Testing</b> : puedes probar las llamadas a la API de forma independiente. |
| ¿Qué problema puede ocurrir si tu pipeline ejecuta la llamada a la API en un bucle sin límite?   | Puede generar <b>miles de llamadas accidentales</b> , eliminando el crédito gratuito o generando facturas enormes. Es lo que se conoce como "bucle infinito" o "ataque de rate limiting". Se debe implementar control de límites (rate limiting) y monitoreo.                                     |

### BLOQUE 5 — Pregunta de síntesis

Diseña una aplicación real que use una API de IA. Indica: (1) qué problema resuelve, (2) qué proveedor y modelo elegirías, (3) qué parámetros clave usarías (*temperature*, *max\_tokens*, *system prompt*), (4) cómo integrarías la API key de forma segura en un repositorio con pipeline CI/CD. Justifica cada decisión:



## APIs de IA — Conceptos Clave y Llamadas



### 1. Problema que resuelve

**Título:** Asistente de soporte técnico automatizado para tickets de helpdesk

**Descripción:** Un sistema que recibe tickets de soporte técnico, los clasifica automáticamente por categoría y prioridad, y genera una respuesta inicial personalizada para el usuario antes de que un agente humano atienda el caso.

**Beneficio:** Reduce el tiempo de respuesta inicial y permite a los agentes enfocarse en casos complejos.

### 2. Proveedor y modelo elegido

| Decisión                           | Justificación                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Proveedor: OpenAI</b>           | Modelos GPT tienen el mejor rendimiento en comprensión de texto y clasificación.                     |
| <b>Modelo: GPT-4o-mini</b>         | Coste muy bajo (\$0.002/1K tokens). Suficiente para clasificación y generación de respuestas cortas. |
| <b>Alternativa: Claude 3 Haiku</b> | Si se necesita más rapidez y menor coste para tareas masivas.                                        |

### 3. Parámetros clave



## APIs de IA — Conceptos Clave y Llamadas



| Parámetro            | Valor                                                                                                                                                                                                     | Justificación                                                                                                        |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>temperature</b>   | 0.2                                                                                                                                                                                                       | Respuestas consistentes y predecibles. Para clasificación queremos siempre el mismo resultado ante inputs similares. |
| <b>max_tokens</b>    | 300                                                                                                                                                                                                       | Limitamos la longitud de la respuesta inicial. Suficiente para saludar y darnos los próximos pasos.                  |
| <b>system prompt</b> | "Eres un asistente de soporte técnico. Clasifica el ticket en categoría [técnico/facturación/cuenta/otro] y prioridad [alta/media/baja]. Luego genera una respuesta inicial amigable de máximo 2 frases." | Define el rol, las categorías permitidas y el formato de salida.                                                     |
| <b>few-shot</b>      | Incluir 2-3 ejemplos de clasificación en el prompt                                                                                                                                                        | Asegura que el modelo entienda exactamente las categorías.                                                           |

## 4. Integración segura con CI/CD

| Paso              | Acción                                                                                           |
|-------------------|--------------------------------------------------------------------------------------------------|
| 1. GitHub Secrets | Guardar OPENAI_API_KEY en Settings → Secrets and variables → Actions                             |
| 2. Workflow yaml  | Pasar la key como variable de entorno: env:<br>OPENAI_API_KEY:<br>\${{ secrets.OPENAI_API_KEY }} |
| 3. Código         | Usar os.getenv("OPENAI_API_KEY") (nunca hardcodear)                                              |
| 4. .gitignore     | Añadir .env para no subirlo nunca                                                                |
| 5. Rate limiting  | Añadir máximo de 100 llamadas/día en el pipeline                                                 |
| 6. Monitoreo      | Usar dashboard de OpenAI para контроль consumo                                                   |

### Ejemplo de código (Python)

```
# clasificador.py
from openai import OpenAI
import os
import json

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

SYSTEM_PROMPT = """Eres un asistente de soporte técnico.
Clasifica el ticket en categoría [técnico, facturación, cuenta, otro] y prioridad [alta,
media, baja].
Responde SOLO en JSON con este formato:
{"categoría": "...", "prioridad": "...", "respuesta": "..."}"""

def clasificar_ticket(descripcion):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": descripcion}
        ],
        temperature=0.2,
        max_tokens=300
    )
    return json.loads(response.choices[0].message.content)
```



## APIs de IA — Conceptos Clave y Llamadas



### Resumen de decisiones

| Aspecto     | Decisión                            | Por qué                         |
|-------------|-------------------------------------|---------------------------------|
| Problema    | Clasificación automática de tickets | Optimiza el flujo de soporte    |
| Proveedor   | OpenAI                              | Mejor rendimiento en NLP        |
| Modelo      | GPT-4o-mini                         | Mejor equilibrio precio/calidad |
| Temperature | 0.2                                 | Consistencia en clasificación   |
| Max tokens  | 300                                 | Respuestas cortas iniciales     |
| Pipeline    | GitHub Actions + Secrets            | Seguridad en CI/CD              |